

Nexus IRC Chat

Dokumentácia projektu

VPWA Projekt

14. decembra 2025

Obsah

1	Zadanie	3
1.1	Požiadavky na funkcionality	3
1.1.1	Autentifikácia	3
1.1.2	Kanály	3
1.1.3	Príkazy	3
1.1.4	Chat	4
1.1.5	Status a notifikácie	4
1.2	Technologické požiadavky	4
2	Diagram fyzického dátového modelu	5
2.0.1	users	5
2.0.2	channels	5
2.0.3	channel_members	6
2.0.4	messages	6
2.0.5	kicks	6
2.0.6	access_tokens	6
2.0.7	Backend (AdonisJS)	7
2.0.8	Databáza (SQLite)	7
2.0.9	Komunikačný tok	7
2.0.10	Real-time funkcionality	7
3	Návrhové rozhodnutia	8
3.1	Výber frontend frameworku – Quasar	8
3.2	Výber backend frameworku – AdonisJS	8
3.3	Výber databázy – SQLite	8
3.4	State management – Pinia	9
3.5	HTTP klient – Axios	9
3.6	Real-time komunikácia – Polling namiesto WebSocket	9
3.7	Autentifikácia – JWT tokeny	10
3.8	Command parsing – Vlastný parser	10
3.9	Notifikácie – Web Notifications API	10
3.10	Offline podpora – Service Workers	10
4	Snímky obrazoviek	11
4.1	Registračná stránka	11
4.2	Hlavná stránka s kanálmi	11
4.3	Chatovacie rozhranie	12
4.4	Profil používateľa	13
5	Záver	14
5.1	Naplnené požiadavky	14
5.2	Možné budúce rozšírenia	14

Zadanie

Cieľom projektu bolo vytvoriť funkčnú **progressive web app (PWA)** pre textovú komunikáciu v štýle IRC/Slack. Aplikácia sa volá **Nexus** a poskytuje moderné chatovacie prostredie s podporou kanálov, správ, notifikácií a real-time komunikácie.

Požiadavky na funkcionality

1.1.1 Autentifikácia

- Registrácia nových používateľov s validáciou údajov
- Prihlásenie a odhlásenie používateľov
- Správa používateľského profilu (meno, priezvisko, prezývka, email)
- Aktualizácia statusu používateľa (Online, DND, Offline)

1.1.2 Kanály

- Vytváranie verejných a súkromných kanálov
- Správa členov kanála (vstup, odchod, pozvánky)
- Administrátorské práva pre zakladateľa kanála
- Automatické mazanie neaktívnych kanálov (30 dní nečinnosti)
- Systém banovania používateľov (3 hlasy alebo okamžitý ban od admina)

1.1.3 Príkazy

Aplikácia podporuje nasledujúce príkazy:

- `/join channelName [private]` – vstup alebo vytvorenie kanála
- `/invite nickName` – pozvanie používateľa do kanála
- `/revoke nickName` – odvolanie pozvánky
- `/kick nickName` – hlasovanie o vyhodení (3 hlasy = ban)
- `/quit` – admin ruší kanál
- `/cancel` – používateľ opúšťa kanál
- `/list` – zoznam členov kanála
- `/help` – zoznam dostupných príkazov

1.1.4 Chat

- Real-time odosielanie a prijímanie správ
- Mentions pomocou @nickname s automatickým zvýraznením
- História správ s podporou infinite scroll
- Indikátor písania (typing indicator)
- Živý náhľad rozpísaného textu (draft messages)

1.1.5 Status a notifikácie

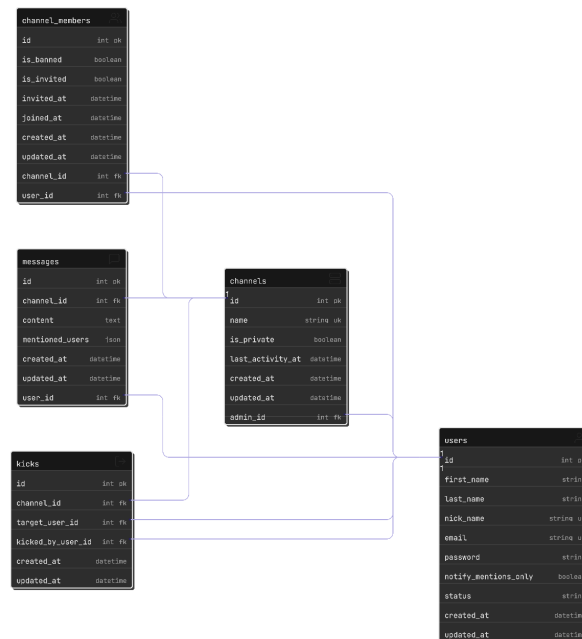
- Status používateľa: Online, DND (Do Not Disturb), Offline
- Web Notifications API pre notifikácie
- Notifikácie len keď aplikácia nie je viditeľná
- Mentions-only režim notifikácií
- Žiadne notifikácie pri DND stave
- Offline režim – správy sa nedoručujú, po návrate sa zosynchronizujú

Technologické požiadavky

- Frontend: Quasar Framework (Vue 3) s PWA podporou
- Backend: AdonisJS framework
- Databáza: SQLite
- State management: Pinia
- HTTP klient: Axios
- Real-time komunikácia: Polling mechanizmus

Diagram fyzického dátového modelu

Databázový model aplikácie Nexus pozostáva z nasledujúcich tabuliek a vzťahov:



2.0.1 users

Hlavná tabuľka pre používateľov aplikácie. Obsahuje základné informácie o používateľovi a jeho nastavenia.

- id – primárny kľúč
- first_name, last_name – meno a priezvisko
- nick_name – unikátna prezývka používateľa
- email – unikátny email
- password – hash hesla
- status – aktuálny status (Online, DND, Offline)
- notify_mentions_only – nastavenie notifikácií

2.0.2 channels

Tabuľka pre chat kanály. Podporuje verejné aj súkromné kanály.

- id – primárny kľúč
- name – unikátny názov kanála
- is_private – flag pre súkromný kanál
- admin_id – cudzí kľúč na používateľa (zakladateľ)
- last_activity_at – čas poslednej aktivity (pre automatické mazanie)

2.0.3 channel_members

Junction tabuľka pre vzťah many-to-many medzi používateľmi a kanálmi.

- `id` – primárny kľúč
- `channel_id` – cudzí kľúč na kanál
- `user_id` – cudzí kľúč na používateľa
- `is_banned` – flag pre zabanovaného používateľa
- `is_invited` – flag pre pozvaného používateľa
- `invited_at`, `joined_at` – časové značky
- Unikátny constraint na (`channel_id`, `user_id`)

2.0.4 messages

Tabuľka pre správy v kanáloch.

- `id` – primárny kľúč
- `channel_id` – cudzí kľúč na kanál
- `user_id` – cudzí kľúč na autora správy
- `content` – obsah správy
- `mentioned_users` – JSON pole s ID používateľov spomenutých v správe
- Index na (`channel_id`, `created_at`) pre rýchle načítanie správ

2.0.5 kicks

Tabuľka pre hlasovanie o vyhodení používateľov z kanála.

- `id` – primárny kľúč
- `channel_id` – cudzí kľúč na kanál
- `target_user_id` – cudzí kľúč na vyhodeného používateľa
- `kicked_by_user_id` – cudzí kľúč na používateľa, ktorý hlasoval
- Unikátny constraint na (`channel_id`, `target_user_id`, `kicked_by_user_id`)

2.0.6 access_tokens

Tabuľka pre JWT tokeny používané pri autentifikácii (súčasť AdonisJS Auth).

2.0.7 Backend (AdonisJS)

Backend je postavený na AdonisJS frameworku:

- **Controllers** – obsluha HTTP požiadaviek
- **Models (Lucid ORM)** – databázové modely a vzťahy
- **Middleware** – autentifikácia, CORS, JSON responses
- **Validators** – validácia vstupných dát
- **Routes** – definícia API endpointov

2.0.8 Databáza (SQLite)

SQLite databáza poskytuje:

- Perzistentné ukladanie dát
- Transakčnú podporu
- Referenčnú integritu cez foreign keys
- Indexy pre optimalizáciu výkonu

2.0.9 Komunikačný tok

1. Frontend posiela HTTP požiadavku na backend API
2. Backend validuje požiadavku a autentifikáciu
3. Backend vykonáva databázové dotazy cez Lucid ORM
4. Backend vracia JSON odpoveď
5. Frontend aktualizuje stav cez Pinia stores
6. UI sa re-renderuje na základe nového stavu

2.0.10 Real-time funkcionality

Namiesto WebSocketov sa používa polling mechanizmus:

- Frontend periodicky dotazuje backend na nové správy
- Typing indicators sa aktualizujú cez polling
- Draft messages sa synchronizujú cez polling
- Web Notifications API pre notifikácie v prehliadači

Návrhové rozhodnutia

V tejto sekcii sú zdôvodnené kľúčové návrhové rozhodnutia pri vývoji aplikácie Nexus.

Výber frontend frameworku – Quasar

Rozhodnutie: Použitie Quasar frameworku namiesto čistého Vue 3 alebo iných frameworkov.

Zdôvodnenie:

- **PWA podpora out-of-the-box** – Quasar má vstavanú podporu pre Progressive Web Apps, čo výrazne zjednodušuje konfiguráciu service workers a manifest súborov
- **Material Design komponenty** – bohatá sada UI komponentov šetrí čas pri vývoji
- **Cross-platform** – možnosť ľahkého portovania na mobilné platformy v budúcnosti
- **TypeScript podpora** – plná podpora TypeScriptu pre type safety
- **Výkon** – optimalizované komponenty a build proces

Výber backend frameworku – AdonisJS

Rozhodnutie: Použitie AdonisJS namiesto Express.js, NestJS alebo iných Node.js frameworkov.

Zdôvodnenie:

- **TypeScript first** – nativná podpora TypeScriptu bez potreby konfigurácie
- **Lucid ORM** – výkonný ORM s podporou migrácií a seedov
- **Built-in Auth** – vstavaný autentifikačný systém s JWT tokenmi
- **Validators** – robustný systém validácie vstupných dát (VineJS)
- **Convention over configuration** – jasná štruktúra projektu a best practices
- **Active development** – moderný framework s aktívnym vývojom

Výber databázy – SQLite

Rozhodnutie: Použitie SQLite namiesto PostgreSQL alebo MySQL.

Zdôvodnenie:

- **Jednoduchosť** – žiadna potreba samostatného databázového servera
- **Perfektné pre prototyp** – rýchle nastavenie a vývoj
- **File-based** – databáza je jednoduchý súbor, ľahké zálohovanie
- **Výkon** – dostatočný výkon pre malé až stredné aplikácie
- **SQL štandard** – možnosť ľahkého migrovania na PostgreSQL v produkcii
- **Zero configuration** – funguje out-of-the-box

State management – Pinia

Rozhodnutie: Použitie Pinia namiesto Vuex alebo Context API.

Zdôvodnenie:

- **Oficiálny Vue 3 state manager** – odporúčaný nástupca Vuex
- **TypeScript podpora** – výborná podpora TypeScriptu
- **Jednoduchšia syntax** – menej boilerplate kódu než Vuex
- **Composition API** – prirodzená integrácia s Composition API
- **DevTools** – výborné nástroje pre debugging

HTTP klient – Axios

Rozhodnutie: Použitie Axios namiesto natívneho fetch API.

Zdôvodnenie:

- **Interceptors** – možnosť interceptovať requesty a response (pre auth tokeny)
- **Automatická JSON konverzia** – automatická serializácia/deserializácia
- **Error handling** – lepšie error handling než fetch
- **Request cancellation** – možnosť zrušiť požiadavky
- **Browser compatibility** – lepšia podpora starších prehliadačov

Real-time komunikácia – Polling namiesto WebSocket

Rozhodnutie: Použitie polling mechanizmu namiesto WebSocketov.

Zdôvodnenie:

- **Jednoduchšia implementácia** – polling je jednoduchšie implementovať a debugovať
- **HTTP kompatibilita** – funguje cez štandardné HTTP, žiadne špeciálne servery
- **Firewall friendly** – funguje aj cez firewally, ktoré blokujú WebSockets
- **Pre prototyp postačujúce** – pre MVP je polling dostatočný
- **Možnosť upgrade** – v budúcnosti možno ľahko migrovať na WebSocket
- **Offline handling** – jednoduchšie spracovanie offline stavov

Autentifikácia – JWT tokeny

Rozhodnutie: Použitie JWT (JSON Web Tokens) pre autentifikáciu.

Zdôvodnenie:

- **Stateless** – server nemusí ukladať session stav
- **Scalability** – ľahšie škálovanie, žiadne session storage
- **Cross-domain** – funguje aj cez rôzne domény
- **AdonisJS podpora** – vstavaná podpora v AdonisJS Auth
- **Bezpečnosť** – tokeny sú podpísané a možno šifrovať

Command parsing – Vlastný parser

Rozhodnutie: Vytvorenie vlastného command parsera namiesto použitia existujúcej knižnice.

Zdôvodnenie:

- **Jednoduchosť** – príkazy sú jednoduché, netreba komplexnú knižnicu
- **Customizácia** – plná kontrola nad parsovaním a validáciou
- **Bez závislostí** – menej externých knižníc
- **TypeScript** – možnosť písať type-safe parser

Notifikácie – Web Notifications API

Rozhodnutie: Použitie Web Notifications API namiesto push notifikácií.

Zdôvodnenie:

- **Jednoduchosť** – funguje bez potreby service worker pre základné notifikácie
- **User control** – používateľ má plnú kontrolu cez prehliadač
- **Respektuje DND** – rešpektuje systémové nastavenia
- **No server setup** – nevyžaduje push notification server

Offline podpora – Service Workers

Rozhodnutie: Implementácia offline podpory cez Service Workers.

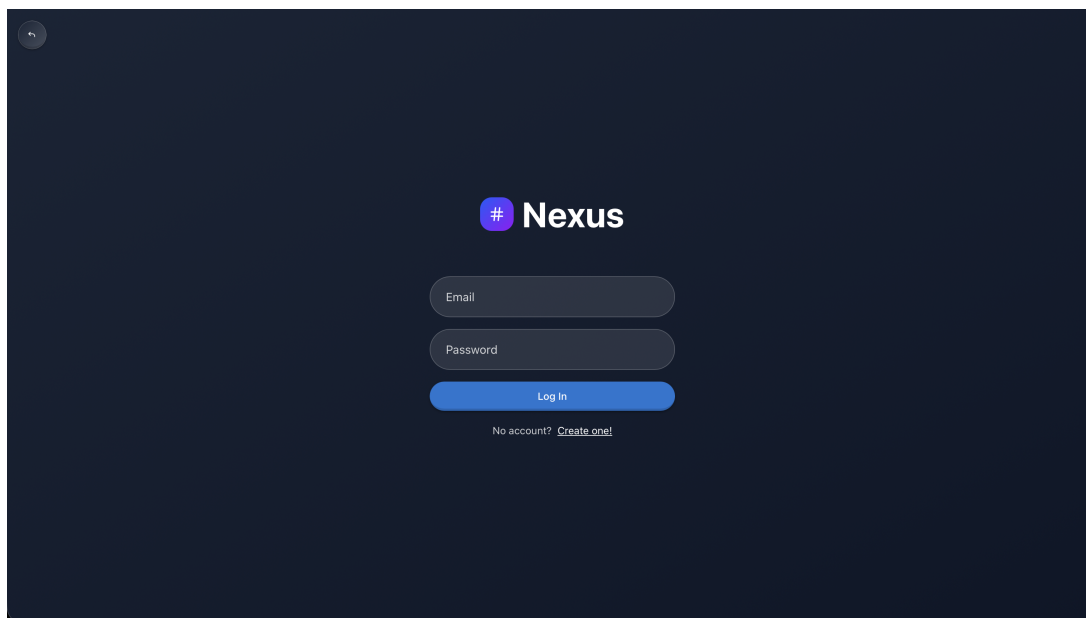
Zdôvodnenie:

- **PWA požiadavka** – základná funkcionálna PWA
- **Quasar podpora** – Quasar automaticky generuje service workers
- **User experience** – aplikácia funguje aj offline
- **Sync mechanism** – možnosť synchronizovať dáta po návrate online

Snímky obrazoviek

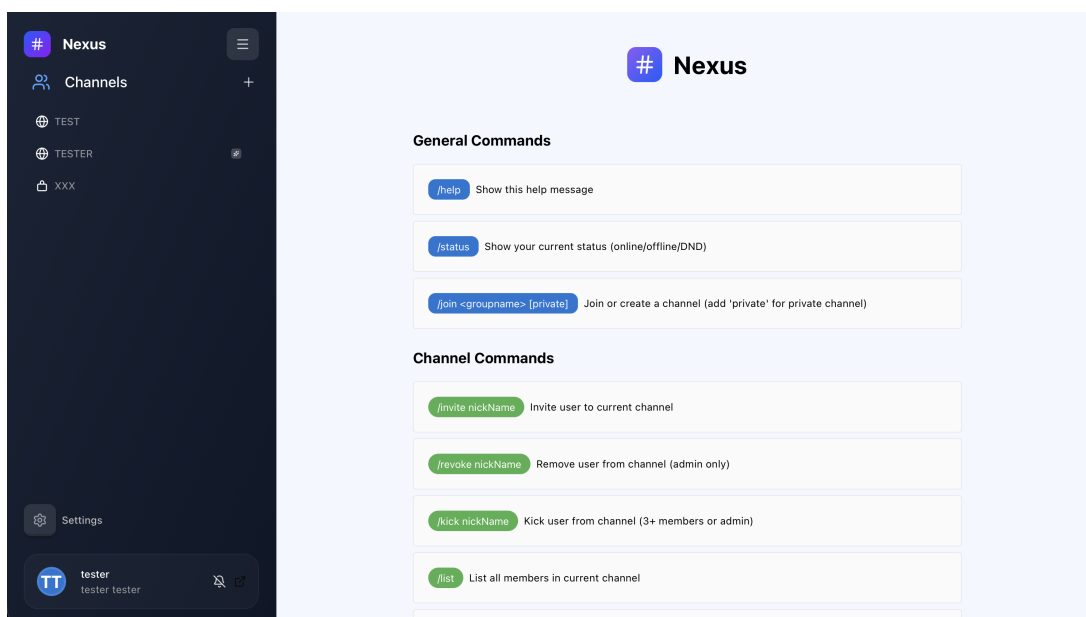
V tejto sekcii sú prezentované kľúčové obrazovky aplikácie Nexus, ktoré demonštrujú hlavnú funkcionálnosť a používateľské rozhranie.

Registračná stránka



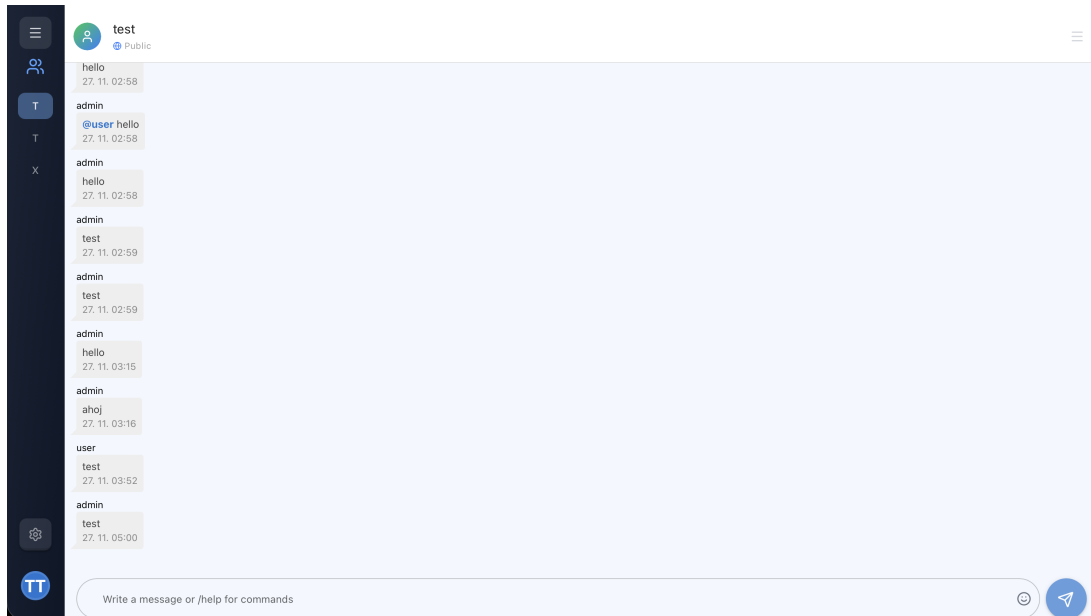
Popis: Registračná stránka umožňuje novým používateľom vytvoriť účet. Formulár obsahuje polia pre meno, priezvisko, prezývku, email a heslo. Aplikácia validuje údaje na strane klienta aj servera a poskytuje jasnú spätnú väzbu používateľovi.

Hlavná stránka s kanálmi



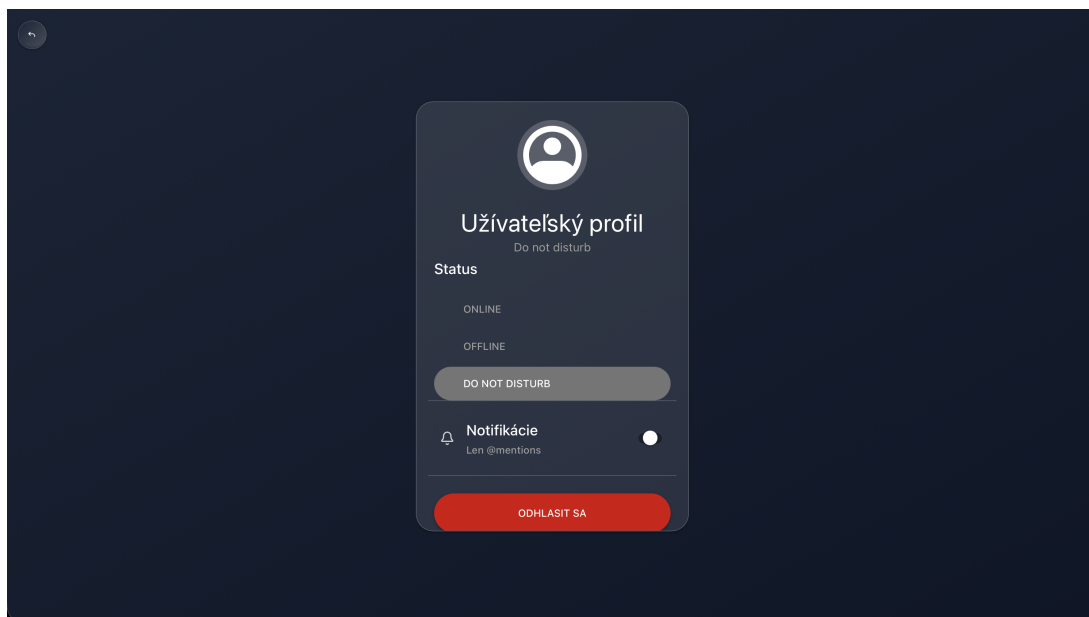
Popis: Hlavná stránka po prihlásení zobrazuje zoznam dostupných kanálov. Používateľ vidí verejné kanály a súkromné kanály, do ktorých bol pozvaný. Stránka obsahuje vyhľadávanie kanálov a tlačidlo pre vytvorenie nového kanála. Zobrazuje sa aj počet členov a posledná aktivita v každom kanáli.

Chatovacie rozhranie



Popis: Hlavné chatovacie rozhranie aplikácie. Zobrazuje históriu správ s podporou infinite scroll, zvýraznenie mentions (@nickname), indikátor písania iných používateľov a input pole pre odosielanie správ. Správy sú zobrazené s informáciami o autorovi a časom odoslania.

Profil používateľa



Popis: Stránka profilu umožňuje používateľovi zmeniť status (Online, DND, Offline). Zobrazuje sa aj nastavenie notifikácií (mentions-only režim). Všetky zmeny sú validované a uložené do databázy.

Záver

Aplikácia Nexus je úspešne implementovaná progressive web app pre textovú komunikáciu v štýle IRC/Slack. Projekt využíva moderné technológie (Quasar, AdonisJS, SQLite) a poskytuje kompletnú funkcionálnu podľa zadania.

Naplnené požiadavky

- Autentifikácia a správa používateľov
- Vytváranie a správa kanálov (verejných aj súkromných)
- Chatovanie s mentions a históriou správ
- Príkazy v chate (/join, /invite, /kick, atď.)
- Status používateľov a notifikácie
- Offline podpora a synchronizácia
- PWA funkcionálna
- Responzívny dizajn

Možné budúce rozšírenia

- WebSocket podpora pre skutočný real-time chat
- Súborové prílohy v správach
- Emoji reactions na správy
- Thread replies (odpovede na konkrétne správy)
- Hľadanie v histórii správ
- Dark mode
- Mobilná aplikácia (Quasar Mobile)
- End-to-end šifrovanie

Technické špecifikácie

- **Frontend:** Quasar Framework v2.16.0, Vue 3.5.20, Pinia 3.0.1
- **Backend:** AdonisJS 6.18.0, Node.js
- **Databáza:** SQLite 3 (better-sqlite3 12.4.6)
- **Autentifikácia:** JWT tokens (AdonisJS Auth)
- **HTTP klient:** Axios 1.2.1
- **TypeScript:** 5.8+ (backend), 5.9.2 (frontend)